Team Resource / Meeting Room Booking System

Candidate Screening Project — Level 2 (~6 months experience) · Prototype scope

1. Overview

Build an internal booking tool for shared resources (meeting rooms, equipment, whatever you want to model) with real conflict-free scheduling, an approval workflow, and in-app real-time notifications. No external calendar integration — this tests whether you can build correct scheduling logic yourself, not whether you can call the Google Calendar API.

2. Functional Requirements

1. **Auth & roles:** users log in via Supabase Auth; at least two roles — admin and regular member.
2. **Resource management:** admins create bookable resources (name, description/capacity, and a flag for whether booking that resource needs admin approval).
3. **Availability view:** any logged-in member can view a resource's booked/free time slots for a given day or week.
4. **Book a slot:** a member selects a start/end time and books it. The system must reject overlapping bookings for the same resource — including when two people submit near-simultaneous requests for the same slot (a real concurrency scenario, not just a client-side check).
5. **Recurring bookings:** support a simple recurrence option, e.g. “every Tuesday 3–4pm for N weeks,” which creates the underlying individual booking records.
6. **Approval workflow:** bookings on “needs approval” resources start in a pending state; an admin can approve or reject from an admin view. Rejected bookings free up the slot.
7. **In-app notifications:** a notification center (e.g. a bell icon with a list) shows booking confirmations, rejections, and conflict errors to the relevant user — delivered in real time while they're in the app, not just on page refresh.
8. **Calendar export (optional but recommended):** a downloadable .ics file for a confirmed booking. This is local file generation from booking data — not a call to any calendar API.

3. Explicitly Out of Scope

- Google Calendar / Outlook sync or OAuth
- Email or SMS reminders
- Payment for resource usage
- Multi-organization / multi-tenant support (single team only)

> *Building any of the above will not earn extra credit — it takes time away from getting the in-scope requirements genuinely solid. Scope judgment is itself part of what's being evaluated.*

4. Non-Functional Requirements

- **Concurrency correctness:** double-booking prevention must be enforced at the database layer (e.g. an exclusion constraint or a transaction with proper locking) — a check-then-insert done only in application code is a race condition and will be tested.
- **Real-time, not polling-in-disguise:** if you claim real-time notifications, they should arrive via a subscription (Supabase Realtime or WebSockets), not a 5-second polling loop dressed up as “real-time.”
- **State machine clarity:** booking status (pending/approved/rejected/cancelled) should be an explicit, enumerable state — not inferred from a combination of nullable fields.

- **RLS-enforced permissions:** a regular member must not be able to approve their own booking by calling the API directly, even if the UI hides that button.

5. Suggested Tech Stack

Versions below were verified current as of July 2026. Substituting equivalents is fine — justify the choice in your README.

- **Framework:** `Next.js 16.2` (App Router, current Active LTS as of July 2026 — do not scaffold on 15.x or the 16.3 preview) with `TypeScript 6.0.3`.
- **Runtime:** `Node.js 24` (current Active LTS). Node 22 is Maintenance-only; don't start a new project on it.
- **Styling:** `Tailwind CSS v4`.
- **Validation:** `Zod 4.4.3` for all form and API-boundary validation — never trust client input server-side.
- **Database + Auth:** Supabase — provision on `Postgres 17` (Postgres 14 was deprecated by Supabase on July 1, 2026 and will no longer accept new projects). Use the Supabase CLI (`v2.109+`) for local dev + migrations.
- **Deployment:** Vercel (free tier is enough).
- **Real-time:** `Supabase Realtime` (Postgres change data capture over WebSockets) for the notification center.
- **Overlap prevention:** Postgres `EXCLUDE` constraint using the `btree_gist` extension on (resource_id, time range) — the correct DB-native way to prevent overlapping bookings, rather than hand-written overlap-checking queries.
- **.ics generation:** `ics` npm package (lightweight, no external calls) if you build the calendar export feature.

6. Task List

Phase 0 — Setup

- ☐ Scaffold Next.js 16.2 + TypeScript, Tailwind v4
- ☐ Supabase project on Postgres 17; enable btree_gist extension

Phase 1 — Data Model & Roles

- ☐ Schema: resources, bookings (with a tstzrange time column), notifications
- ☐ Roles: admin vs member (via a role column + RLS policies, not just UI conditionals)
- ☐ EXCLUDE constraint preventing overlapping bookings per resource

Phase 2 — Booking Core

- ☐ Availability view per resource (day/week)
- ☐ Book-a-slot flow, surfacing a clean error when the DB constraint rejects an overlap
- ☐ Recurring booking creation (materializing individual booking rows)

Phase 3 — Approval & Notifications

- ☐ Approval-required resource flag + admin approve/reject view
- ☐ Booking status state machine (pending/approved/rejected/cancelled)
- ☐ Real-time notification center wired to Supabase Realtime

Phase 4 — Extras & Deploy

- ☐ .ics download for a confirmed booking (optional but recommended)

- ☐ Empty/loading/error states throughout
- ☐ Deploy to Vercel; manually test two simultaneous booking attempts on the same slot
- ☐ Write README

7. Submission Requirements

- Live URL
- GitHub repo
- README explaining the conflict-detection approach and how recurring bookings are stored
- A short note on what you'd build next with another week

8. README Must Also Answer

Explain how your system prevents two users from booking the same resource slot at the same time, specifically what happens at the database level, and how recurring bookings are represented (materialized rows vs. a recurrence rule).